Lab 3 – Peripherals Programming on the mbed Part 2: Interfacing the ADXL345 accelerometer module using SPI

Embedded Processors UESTC & UESTCHN 2004 School of Engineering, University of Glasgow

Student Name: Kartike Sharan

Student Number: 3008620s

UESTC 2004 Lab Assessment Criteria

Each individual lab will be evaluated using the below criteria (check the assessment table at the end as they :

- Demonstrating a functional embedded system

  This means each student will need to demonstrate a functional system for each task in the lab manual to the GTA and member of staff to get the full mark.

- Lab Questions (if any)
- Reflection and significance

  After the completion of programming and testing each of the "Tasks", you will be required to reflect, in a few lines, on your understanding of the tasks and its significance. This is a chance for you to ask yourselves "What you did?", "What it means?", and "What next?", meaning what is the significance of the undertaken task in the context of embedded systems and what you are learning in the lecture. Try to be innovative and think outside the box. Please, do not provide a summary of what you did as it will not be accepted.

- Adherence to coding good practice
  - Meaningful variable names – please avoid names like "x", "y", or personal names
  - Using block comments to explain what your code does at the beginning and using line comments to clarify what this line of code does.
  - Consistent indentation – Indentation helps in making the code readable and easy to interpret. See an example of what to do and what to avoid, below:

Good practice ✓

```
#include "mbed.h"
char switch_word ;          //the word we will send
char recd_val;             //the received value
int main() {
```

```
      async_port.baud(9600);      //set baud rate to 9600
      recd_val = 0;
      while (1)
      {
         switch_word=0xa0;
      }
}
```

Avoid this ✗

```
#include "mbed.h"
char switch_word ;         //the word we will send
char recd_val;            //the received value
int main() {
async_port.baud(9600);           //set baud rate to 9600
recd_val = 0;
while (1)
{
switch_word=0xa0;
}
}
```

All the best of luck.

The Embedded Processors Team,

Introduction

The ADXL345 accelerometer is a small, thin, low-power 3-axis accelerometer capable of measuring acceleration up to +/-16g. It uses the SPI protocol to communicate with the host device. In this lab, we will interface the ADXL345 accelerometer to the NUCLEO L432KC mbed board using the SPI interface. We will then print the x/y/z values on the serial terminal and control 8 LEDs based on the x/y values.

Objective

The objective of this lab is to interface the ADXL345 accelerometer to the NUCLEO L432KC mbed board using the SPI interface, read the x/y/z values from the accelerometer, and control LEDs to reflect the orientation of the sensor.

Please note that you need to create separate ".cpp" files for each task and put all together (compressed, e.g., ".zip") and upload them, with your lab exercise where you will have given answers to any questions and written your reflection, to the dedicated moodle link for this lab.

Requirements

Software:

- Keil arm online compiler (https://studio.keil.arm.com/auth/login/) – You will need to create an account to use the online compiler.

Hardware:

For this lab, the following components will be needed:

- NUCLEO L432KC mbed board
- ADXL345 accelerometer
- Breadboard
- Jumper wires
- 8 LEDs (4 red and 4 green)
- USB cable for programming and power

Task 1: Wiring and Hardware Setup

With reference to Figure Task 1, wire the mbed, accelerometer, and LEDs as per the suggested pin connections, below.

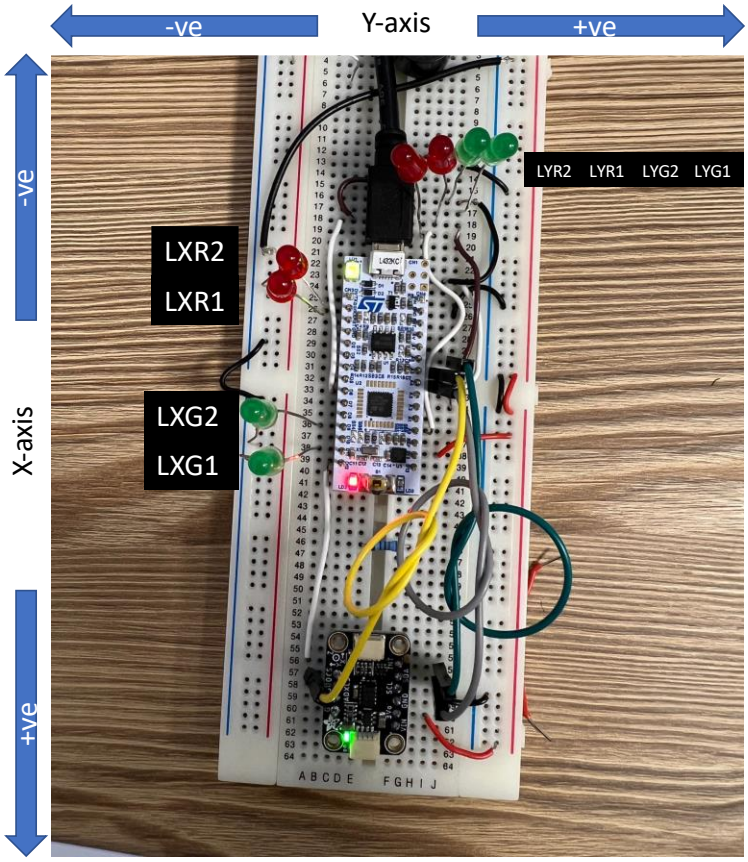
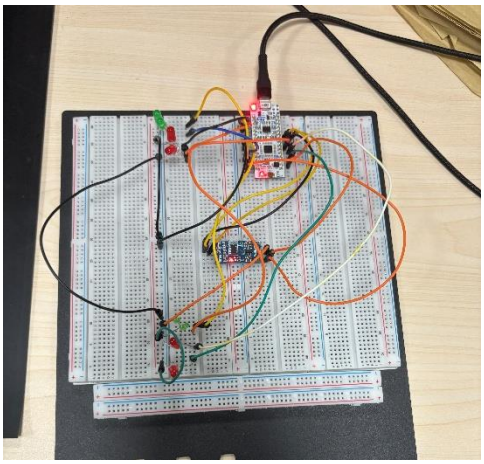


Figure Task 1

Pin connections

Please refer to Figure Task 1 for the LED connections and orientation

SCLK	A4
MISO	A5
MOSI	A6
CS	D12
LXG1	D11
LXG2	D9
LXR1	D0
LXR2	D1
LYG1	D6
LYG2	A3
LYR1	A2



LYR2	A0

Task 2: Code Setup & SPI Initialisation

1. Create a new mbed project in the online compiler or desktop IDE.
2. Import the mbed library.
3. ***#define*** the SPI pins using meaningful names e.g. ***SPI_MOSI***
4. Create the SPI, and output objects.
5. Create the ***ADXL345_SPI_Initialise*** function with following specs:
    - 8 bit data, mode 3
    - 2 MHz clock rate
    - Configure the ADXL345 as follows (consult the datasheet and the lecture slides):
        - 4-wire SPI mode, FULL_RES mode, right-justified mode, +/-16g
        - Link bit cleared, AUTO_SLEEP disabled, measurement mode, normal mode, 8 Hz sleep mode frequency
    - Once finished, call this function in the main code to initialise the accelerometer.

```
/*
------------------------------------------------------------
UESTC 2004 Embedded Processors
Lab 3 - Task 2
ADXL345 SPI Initialisation
------------------------------------------------------------
*/

#include "mbed.h"

// ----------------------------------------------------
// SPI Pin Definitions
// ----------------------------------------------------
#define SPI_MOSI A6
#define SPI_MISO A5
#define SPI_SCLK A4
#define SPI_CS   D12

// ----------------------------------------------------
// ADXL345 Register Addresses
// ----------------------------------------------------
#define DEVID_REG       0x00
#define POWER_CTL_REG   0x2D
#define DATA_FORMAT_REG 0x31
#define BW_RATE_REG     0x2C

// ----------------------------------------------------
// SPI Object
```

```
// ----------------------------------------------------
SPI adxl345_spi(SPI_MOSI, SPI_MISO, SPI_SCLK);

// Chip Select Pin
DigitalOut cs(SPI_CS);

// ----------------------------------------------------
// Function: Write to ADXL345 Register
// ----------------------------------------------------
void ADXL345_Write(unsigned char address, unsigned char value)
{
    cs = 0;                 // Select device
    adxl345_spi.write(address);   // Send register address
    adxl345_spi.write(value);     // Send data
    cs = 1;                 // Deselect device
}

// ----------------------------------------------------
// Function: Initialise ADXL345
// ----------------------------------------------------
void ADXL345_SPI_Initialise()
{
    // SPI format:
    // 8-bit data, Mode 3
    adxl345_spi.format(8, 3);

    // 2 MHz SPI clock
    adxl345_spi.frequency(2000000);

    cs = 1;

    // ----------------------------------------------------
    // DATA_FORMAT Register (0x31)
    //
    // FULL_RES = 1
    // Justify = 0 (right justified)
    // Range = 11 (+/-16g)
    // SPI = 0 (4-wire SPI)
    //
    // Binary: 00001011 = 0x0B
    // ----------------------------------------------------
    ADXL345_Write(DATA_FORMAT_REG, 0x0B);

    // ----------------------------------------------------
    // POWER_CTL Register (0x2D)
    //
    // Link = 0
    // AUTO_SLEEP = 0
    // Measure = 1
```

```
    // Sleep = 0
    //
    // Binary: 00001000 = 0x08
    // ----------------------------------------------------
    ADXL345_Write(POWER_CTL_REG, 0x08);

    // ----------------------------------------------------
    // BW_RATE Register (0x2C)
    //
    // Normal mode
    // Output data rate = 8 Hz
    //
    // Binary: 00001000 = 0x08
    // ----------------------------------------------------
    ADXL345_Write(BW_RATE_REG, 0x08);
}

// ----------------------------------------------------
// Main Program
// ----------------------------------------------------
int main()
{
    printf("Initialising ADXL345...\n");

    ADXL345_SPI_Initialise();

    printf("ADXL345 Initialised Successfully\n");

    while (1)
    {
        // Do nothing for Task 2
    }
}
```

Task 3: Reading Data from the ADXL345

1. In the main code, implement a loop to continuously read the accelerometer data
2. Use the SPI interface to read the raw data from the accelerometer into a buffer array
3. Convert the raw data into 16-bit two's complement integers and then into floating-point values for each axis (X, Y, and Z)
4. Print the X, Y, and Z values on the serial terminal
5. Tilt the breadboard to see the change in values and show it to the lab supervisor

```
/*
-----------------------------------------------------
UESTC 2004 Embedded Processors
Lab 3 - Task 3
ADXL345 SPI Reading + Serial Output
-----------------------------------------------------
*/

#include "mbed.h"

// ---------------------------------------------------
// SPI Pin Definitions (NUCLEO-L432KC)
// ---------------------------------------------------
#define ADXL_MOSI A6
#define ADXL_MISO A5
#define ADXL_SCLK A4
#define ADXL_CS D12

// ---------------------------------------------------
// ADXL345 Register Addresses
// ---------------------------------------------------
#define POWER_CTL_REG 0x2D
#define DATA_FORMAT_REG 0x31
#define BW_RATE_REG 0x2C
#define DATAX0_REG 0x32

// ---------------------------------------------------
// SPI Object
// ---------------------------------------------------
SPI adxl345_spi(ADXL_MOSI, ADXL_MISO, ADXL_SCLK);

// Chip Select
DigitalOut cs(ADXL_CS);

// ---------------------------------------------------
// Write to ADXL345 Register
// ---------------------------------------------------
```

```
void ADXL345_Write(char address, char value)
{
    cs = 0;
    adxl345_spi.write(address);
    adxl345_spi.write(value);
    cs = 1;
}

// ----------------------------------------------------
// Read Multiple Bytes from ADXL345
// ----------------------------------------------------
void ADXL345_Read(char startAddress, char* buffer, int size)
{
    // Read bit + multi-byte bit
    char address = 0x80 | 0x40 | startAddress;

    cs = 0;
    adxl345_spi.write(address);

    for (int i = 0; i < size; i++)
    {
        buffer[i] = adxl345_spi.write(0x00);
    }
    cs = 1;
}

// ----------------------------------------------------
// Initialise ADXL345
// ----------------------------------------------------
void ADXL345_SPI_Initialise()
{
    // SPI format: 8-bit data, SPI mode 3
    adxl345_spi.format(8, 3);

    // SPI frequency = 2 MHz
    adxl345_spi.frequency(2000000);

    // CS inactive high
    cs = 1;

    // DATA_FORMAT: FULL_RES + +/-16g
    ADXL345_Write(DATA_FORMAT_REG, 0x0B);

    // POWER_CTL: Measurement mode
    ADXL345_Write(POWER_CTL_REG, 0x08);

    // BW_RATE: 25 Hz output rate
```

```
    ADXL345_Write(BW_RATE_REG, 0x08);
}

// ------------------------------------------------
// Main Program
// ------------------------------------------------
int main()
{
    char data[6];
    int16_t xRaw, yRaw, zRaw;
    float x, y, z;

    printf("ADXL345 Starting...\n");

    // Initialise accelerometer
    ADXL345_SPI_Initialise();

    printf("Initialisation Complete\n");

    while (1)
    {
        // Read 6 bytes starting from DATAX0
        ADXL345_Read(DATAX0_REG, data, 6);

        // Combine bytes into signed 16-bit values
        xRaw = (data[1] << 8) | data[0];
        yRaw = (data[3] << 8) | data[2];
        zRaw = (data[5] << 8) | data[4];

        // Convert to acceleration in g (3.9 mg per LSB for ±16g mode)
        x = xRaw * 0.0039f;
        y = yRaw * 0.0039f;
        z = zRaw * 0.0039f;

        // Print in milli-g
        printf("X = %d mg   Y = %d mg   Z = %d mg\n",
               (int)(x * 1000), (int)(y * 1000), (int)(z * 1000));

        // Delay 200 ms
        thread_sleep_for(200);
    }

}
```

```
X = -93 mg    Y = -222 mg    Z = 1357 mg
X = 230 mg    Y = -397 mg    Z = 678 mg
X = 265 mg    Y = -737 mg    Z = 861 mg
X = -222 mg    Y = 46 mg    Z = 900 mg
X = -245 mg    Y = 943 mg    Z = 421 mg
X = -460 mg    Y = 503 mg    Z = 896 mg
X = -35 mg    Y = -113 mg    Z = 1595 mg
X = 62 mg    Y = 66 mg    Z = 1068 mg
X = -608 mg    Y = 136 mg    Z = 904 mg
X = -425 mg    Y = 159 mg    Z = 924 mg
X = 155 mg    Y = 58 mg    Z = 1333 mg
X = -148 mg    Y = 257 mg    Z = 1092 mg
X = -510 mg    Y = 101 mg    Z = 908 mg
X = -11 mg    Y = 269 mg    Z = 947 mg
```

Task 4: LED Control using the ADXL345

Refer to Figure Task 1 for LED positioning and orientation!

1. Based on the value of the X-axis reading, control the state of the appropriate LEDs to reflect the current orientation of the accelerometer
2. If X is between -0.05 and -0.5, turn on LXR1, turn off all others
3. If X is less than -0.5, turn on LXR2, turn off all others
4. If X is between 0.05 and 0.5, turn on LXG2, turn off all others
5. If X is greater than 0.5, turn on LXG1, turn off all others
6. Repeat steps 1 to 5 for the Y-axis readings to control the state of the remaining four LEDs
7. Upload the completed code to the mbed microcontroller
8. Test the functionality of the accelerometer-based LED control by tilting the accelerometer in various directions and observing the corresponding LED states
9. Debug any issues or errors that arise in the code or wiring

```
/*
-----------------------------------------------------
UESTC 2004 Embedded Processors
Lab 3 - Task 4
ADXL345 SPI + LED Control based on X and Y tilt
-----------------------------------------------------
*/

#include "mbed.h"

// ---------------------------------------------------
// SPI Pin Definitions
// ---------------------------------------------------
#define ADXL_MOSI A6
#define ADXL_MISO A5
#define ADXL_SCLK A4
#define ADXL_CS D12

// ---------------------------------------------------
// LED Pin Definitions (from Figure Task 1)
// ---------------------------------------------------
// X-axis LEDs
DigitalOut lxr1(D0);   // LXR1
DigitalOut lxr2(D1);   // LXR2
DigitalOut lxg1(D11);  // LXG1
DigitalOut lxg2(D9);   // LXG2

// Y-axis LEDs
DigitalOut lyr1(A2);   // LYR1
DigitalOut lyr2(A0);   // LYR2
```

```
DigitalOut lyg1(D6);   // LYG1
DigitalOut lyg2(A3);   // LYG2

// ----------------------------------------------------
// ADXL345 Register Addresses
// ----------------------------------------------------
#define POWER_CTL_REG   0x2D
#define DATA_FORMAT_REG 0x31
#define BW_RATE_REG     0x2C
#define DATAX0_REG      0x32

// ----------------------------------------------------
// SPI Object and Chip Select
// ----------------------------------------------------
SPI adxl345_spi(ADXL_MOSI, ADXL_MISO, ADXL_SCLK);
DigitalOut cs(ADXL_CS);

// ----------------------------------------------------
// Write to ADXL345 Register
// ----------------------------------------------------
void ADXL345_Write(char address, char value) {
    cs = 0;
    adxl345_spi.write(address);
    adxl345_spi.write(value);
    cs = 1;
}

// ----------------------------------------------------
// Read Multiple Bytes from ADXL345
// ----------------------------------------------------
void ADXL345_Read(char startAddress, char* buffer, int size) {
    char address = 0x80 | 0x40 | startAddress;  // read + multi-byte
    cs = 0;
    adxl345_spi.write(address);
    for (int i = 0; i < size; i++) {
        buffer[i] = adxl345_spi.write(0x00);
    }
    cs = 1;
}

// ----------------------------------------------------
// Initialise ADXL345
// ----------------------------------------------------
void ADXL345_SPI_Initialise() {
    adxl345_spi.format(8, 3);       // 8-bit, mode 3
    adxl345_spi.frequency(2000000); // 2 MHz
    cs = 1;
```

```
    ADXL345_Write(DATA_FORMAT_REG, 0x0B);  // FULL_RES, ±16g
    ADXL345_Write(POWER_CTL_REG, 0x08);    // Measurement mode
    ADXL345_Write(BW_RATE_REG, 0x08);      // 25 Hz output rate
}

// ------------------------------------------------
// Turn off all LEDs (useful before setting new state)
// ------------------------------------------------
void all_leds_off() {
    lxr1 = 0; lxr2 = 0; lxg1 = 0; lxg2 = 0;
    lyr1 = 0; lyr2 = 0; lyg1 = 0; lyg2 = 0;
}

// ------------------------------------------------
// Control LEDs based on X and Y values (in g)
// ------------------------------------------------
void control_leds(float x_g, float y_g) {
    // --- X-axis control (mutually exclusive) ---
    if (x_g < -0.5f) {
        // X less than -0.5g → LXR2 on, all other X LEDs off
        lxr2 = 1; lxr1 = 0; lxg1 = 0; lxg2 = 0;
    }
    else if (x_g < -0.05f) {
        // X between -0.5 and -0.05 → LXR1 on
        lxr1 = 1; lxr2 = 0; lxg1 = 0; lxg2 = 0;
    }
    else if (x_g > 0.5f) {
        // X greater than 0.5g → LXG1 on
        lxg1 = 1; lxr1 = 0; lxr2 = 0; lxg2 = 0;
    }
    else if (x_g > 0.05f) {
        // X between 0.05 and 0.5 → LXG2 on
        lxg2 = 1; lxr1 = 0; lxr2 = 0; lxg1 = 0;
    }
    else {
        // -0.05 ≤ X ≤ 0.05 → all X LEDs off
        lxr1 = 0; lxr2 = 0; lxg1 = 0; lxg2 = 0;
    }

    // --- Y-axis control (mutually exclusive) ---
    if (y_g < -0.5f) {
        lyr2 = 1; lyr1 = 0; lyg1 = 0; lyg2 = 0;
    }
    else if (y_g < -0.05f) {
        lyr1 = 1; lyr2 = 0; lyg1 = 0; lyg2 = 0;
    }
```

```
    else if (y_g > 0.5f) {
        lyg1 = 1; lyr1 = 0; lyr2 = 0; lyg2 = 0;
    }
    else if (y_g > 0.05f) {
        lyg2 = 1; lyr1 = 0; lyr2 = 0; lyg1 = 0;
    }
    else {
        lyr1 = 0; lyr2 = 0; lyg1 = 0; lyg2 = 0;
    }
}

// ------------------------------------------------
// Main Program
// ------------------------------------------------
int main() {
    char data[6];
    int16_t xRaw, yRaw, zRaw;
    float x, y, z;

    printf("ADXL345 Starting...\n");
    ADXL345_SPI_Initialise();
    printf("Initialisation Complete\n");

    all_leds_off();  // start with all LEDs off

    while (1) {
        // Read 6 bytes from DATAX0 register
        ADXL345_Read(DATAX0_REG, data, 6);

        xRaw = (data[1] << 8) | data[0];
        yRaw = (data[3] << 8) | data[2];
        zRaw = (data[5] << 8) | data[4];

        // Convert to g (3.9 mg per LSB for ±16g mode)
        x = xRaw * 0.0039f;
        y = yRaw * 0.0039f;
        z = zRaw * 0.0039f;

        // Print values (optional, for debugging)
        printf("X = %d mg   Y = %d mg   Z = %d mg\n",
               (int)(x * 1000), (int)(y * 1000), (int)(z * 1000));

        // Update LEDs based on X and Y tilt
        control_leds(x, y);

        thread_sleep_for(200);  // 200 ms delay
    }
```

```
}
```

Task 5: Further Exploration (Bonus Task worth 10%)

- Modify the code to control the LEDs based on a combination of X and Y-axis readings.

```
/*
------------------------------------------------------
UESTC 2004 Embedded Processors
Lab 3 - Task 5 (Bonus)
ADXL345 SPI + Combined X/Y LED Control
Uses tilt angle and magnitude to drive LED combinations
------------------------------------------------------
*/

#include "mbed.h"
#include <cmath>   // for atan2f, sqrtf, fabsf

// ------------------------------------------------
// SPI Pin Definitions
// ------------------------------------------------
#define ADXL_MOSI A6
#define ADXL_MISO A5
#define ADXL_SCLK A4
#define ADXL_CS   D12

// ------------------------------------------------
// LED Pin Definitions
// ------------------------------------------------
DigitalOut led_x_red1(D0);    // LXR1 – mild -X tilt
DigitalOut led_x_red2(D1);    // LXR2 – strong -X tilt
DigitalOut led_x_grn1(D11);   // LXG1 – strong +X tilt
DigitalOut led_x_grn2(D9);    // LXG2 – mild +X tilt

DigitalOut led_y_red1(A2);    // LYR1 – mild -Y tilt
DigitalOut led_y_red2(A0);    // LYR2 – strong -Y tilt
DigitalOut led_y_grn1(D6);    // LYG1 – strong +Y tilt
DigitalOut led_y_grn2(A3);    // LYG2 – mild +Y tilt

// ------------------------------------------------
// ADXL345 Register Addresses
// ------------------------------------------------
#define POWER_CTL_REG   0x2D
#define DATA_FORMAT_REG 0x31
#define BW_RATE_REG     0x2C
#define DATAX0_REG      0x32

// ------------------------------------------------
// Tilt thresholds (in g)
// ------------------------------------------------
#define DEAD_ZONE_THRESHOLD  0.05f   // below this → all LEDs off
```

```
#define MILD_TILT_THRESHOLD  0.35f   // mild tilt (1 LED per axis)
#define STRONG_TILT_THRESHOLD 0.65f  // strong tilt (2 LEDs per axis)

// ----------------------------------------------------
// SPI and Chip Select
// ----------------------------------------------------
SPI adxl345_spi(ADXL_MOSI, ADXL_MISO, ADXL_SCLK);
DigitalOut chip_select(ADXL_CS);

// ----------------------------------------------------
// Write to ADXL345 register
// ----------------------------------------------------
void ADXL345_Write(char reg_address, char reg_value) {
    chip_select = 0;
    adxl345_spi.write(reg_address);
    adxl345_spi.write(reg_value);
    chip_select = 1;
}

// ----------------------------------------------------
// Read multiple bytes from ADXL345
// ----------------------------------------------------
void ADXL345_Read(char start_address, char* data_buffer, int byte_count) {
    char read_address = 0x80 | 0x40 | start_address; // read + multi-byte flags
    chip_select = 0;
    adxl345_spi.write(read_address);
    for (int byte_index = 0; byte_index < byte_count; byte_index++) {
        data_buffer[byte_index] = adxl345_spi.write(0x00);
    }
    chip_select = 1;
}

// ----------------------------------------------------
// Initialise ADXL345
// ----------------------------------------------------
void ADXL345_SPI_Initialise() {
    adxl345_spi.format(8, 3);         // 8-bit data, SPI mode 3
    adxl345_spi.frequency(2000000);   // 2 MHz clock
    chip_select = 1;                  // deselect device initially

    // FULL_RES, right-justified, ±16g range, 4-wire SPI
    ADXL345_Write(DATA_FORMAT_REG, 0x0B);

    // Measurement mode, link cleared, auto-sleep disabled, normal power
    ADXL345_Write(POWER_CTL_REG, 0x08);

    // 25 Hz output data rate, normal power mode
    ADXL345_Write(BW_RATE_REG, 0x08);
}
```

```

// ------------------------------------------------
// Turn off all LEDs
// ------------------------------------------------
void all_leds_off() {
    led_x_red1 = 0; led_x_red2 = 0;
    led_x_grn1 = 0; led_x_grn2 = 0;
    led_y_red1 = 0; led_y_red2 = 0;
    led_y_grn1 = 0; led_y_grn2 = 0;
}

// ------------------------------------------------
// Task 5: Combined X+Y LED control
//
// Strategy:
//   1. Compute 2D tilt magnitude from X and Y components
//   2. If below dead zone → all LEDs off
//   3. Compute tilt angle using atan2(Y, X)
//   4. Project the vector onto each axis to determine
//      how much of the tilt is in X and how much in Y
//   5. Drive each axis's LEDs based on their projected
//      component, with mild/strong intensity levels
//
// This means diagonal tilts light up BOTH X and Y LEDs
// simultaneously, which is the key difference from Task 4.
// ------------------------------------------------
void combined_led_control(float x_accel_g, float y_accel_g) {

    // Step 1: compute total tilt magnitude
    float tilt_magnitude = sqrtf(x_accel_g * x_accel_g + y_accel_g * y_accel_g);

    // Step 2: dead zone check – flat on table → all off
    if (tilt_magnitude < DEAD_ZONE_THRESHOLD) {
        all_leds_off();
        return;
    }

    // Step 3: compute angle of tilt vector (radians, -π to +π)
    float tilt_angle_rad = atan2f(y_accel_g, x_accel_g);

    // Step 4: project vector onto X and Y axes
    // These are simply the original readings, but we re-derive
    // them from magnitude × cos/sin for clarity
    float x_component = tilt_magnitude * cosf(tilt_angle_rad); // same as x_accel_g
    float y_component = tilt_magnitude * sinf(tilt_angle_rad); // same as y_accel_g

    // Step 5a: X-axis LED control based on X component
    float abs_x = fabsf(x_component);

```

```
        if (x_component < -STRONG_TILT_THRESHOLD) {
            // Strong tilt toward -X: both red LEDs on
            led_x_red1 = 1; led_x_red2 = 1;
            led_x_grn1 = 0; led_x_grn2 = 0;
        }
        else if (x_component < -MILD_TILT_THRESHOLD) {
            // Mild tilt toward -X: one red LED
            led_x_red1 = 1; led_x_red2 = 0;
            led_x_grn1 = 0; led_x_grn2 = 0;
        }
        else if (x_component > STRONG_TILT_THRESHOLD) {
            // Strong tilt toward +X: both green LEDs on
            led_x_grn1 = 1; led_x_grn2 = 1;
            led_x_red1 = 0; led_x_red2 = 0;
        }
        else if (x_component > MILD_TILT_THRESHOLD) {
            // Mild tilt toward +X: one green LED
            led_x_grn2 = 1; led_x_grn1 = 0;
            led_x_red1 = 0; led_x_red2 = 0;
        }
        else {
            // X near-zero (mostly Y-axis tilt)
            led_x_red1 = 0; led_x_red2 = 0;
            led_x_grn1 = 0; led_x_grn2 = 0;
        }

        // Step 5b: Y-axis LED control based on Y component
        float abs_y = fabsf(y_component);

        if (y_component < -STRONG_TILT_THRESHOLD) {
            led_y_red1 = 1; led_y_red2 = 1;
            led_y_grn1 = 0; led_y_grn2 = 0;
        }
        else if (y_component < -MILD_TILT_THRESHOLD) {
            led_y_red1 = 1; led_y_red2 = 0;
            led_y_grn1 = 0; led_y_grn2 = 0;
        }
        else if (y_component > STRONG_TILT_THRESHOLD) {
            led_y_grn1 = 1; led_y_grn2 = 1;
            led_y_red1 = 0; led_y_red2 = 0;
        }
        else if (y_component > MILD_TILT_THRESHOLD) {
            led_y_grn2 = 1; led_y_grn1 = 0;
            led_y_red1 = 0; led_y_red2 = 0;
        }
        else {
            led_y_red1 = 0; led_y_red2 = 0;
            led_y_grn1 = 0; led_y_grn2 = 0;
        }
```

```
}

// ------------------------------------------------
// Main Program
// ------------------------------------------------
int main() {
    char raw_data[6];
    int16_t x_raw, y_raw, z_raw;
    float x_accel_g, y_accel_g, z_accel_g;

    printf("ADXL345 Task 5 starting...\n");
    ADXL345_SPI_Initialise();
    printf("Initialisation complete\n");

    all_leds_off();

    while (1) {
        // Read 6 bytes starting at DATAX0 (X0, X1, Y0, Y1, Z0, Z1)
        ADXL345_Read(DATAX0_REG, raw_data, 6);

        // Combine bytes into 16-bit two's complement values
        x_raw = (raw_data[1] << 8) | raw_data[0];
        y_raw = (raw_data[3] << 8) | raw_data[2];
        z_raw = (raw_data[5] << 8) | raw_data[4];

        // Convert to g (3.9 mg per LSB in FULL_RES ±16g mode)
        x_accel_g = x_raw * 0.0039f;
        y_accel_g = y_raw * 0.0039f;
        z_accel_g = z_raw * 0.0039f;

        printf("X=%5dmg  Y=%5dmg  Z=%5dmg\n",
               (int)(x_accel_g * 1000),
               (int)(y_accel_g * 1000),
               (int)(z_accel_g * 1000));

        // Drive LEDs based on combined tilt vector
        combined_led_control(x_accel_g, y_accel_g);

        thread_sleep_for(150);   // 150 ms update rate
    }
}
```

Task 6: Reflection

After the completion of programming and testing of Tasks 1 and 2 please reflect, in a few lines, on your understanding of the tasks and its significance. This is a chance for you to ask yourselves **"What you did?", "What it means?", and "What next?"**, meaning what is the significance of the undertaken task in the context of embedded systems and what you are learning in the lecture. Try to be innovative and think outside the box. Please, do not provide a summary of what you did as it will not be accepted.

[Answer in the below space]

Before this lab, SPI was just another acronym from the lectures. Actually wiring up the ADXL345 and writing the initialisation code made it click in a way the slides didn't quite manage. What struck me was how unforgiving it is — if you get the clock mode wrong or write the register values in the wrong order, the sensor doesn't throw an error, it just silently returns nonsense. That was a bit of a wake-up call compared to programming in Python where something usually tells you what went wrong.

The initialisation process also made me realise how many decisions are baked into just a few lines of code. Choosing the measurement range, the resolution, whether the device is in standby or measurement mode — these aren't arbitrary settings, they have real tradeoffs around power consumption and accuracy that actually matter in a deployed system. It made me think about how much engineering goes into something as small as a fitness tracker, where the accelerometer probably runs at a much lower data rate to save battery, and that's all controlled through exactly this kind of register configuration.

Going forward, I think the most valuable thing I'm taking away is the habit of reading a datasheet properly rather than just copying example code. Understanding what each bit in a register actually does means I could apply the same approach to a completely different sensor or peripheral. That feels like a genuinely useful skill, especially for anything involving real hardware rather than just software.

Lab Score

Marking Criteria	Student's Score
Task 1 Demonstration (10%)	
Task 2 Demonstration (15%)	
Task 3 Demonstration (15%)	
Task 4 Demonstration (25%)	
Reflection (20%)	
Adherence to coding good practice (15%)	
Bonus Task 5 (10%)	